

A Neuro-Symbolic Approach to Malware Behavior Analysis Using Semantic Embeddings and Structured Security Knowledge

Raissa Aydina Putri Bakari
MS Cybersecurity Student
CRA DREAM Research Apprentice
Northeastern University, Khoury College of Computer Sciences
Arlington, VA
bakari.r@northeastern.edu

Research Advisor: Dr. Huihui Wang, Northeastern University

Abstract - Malware threat-intelligence reports contain valuable descriptions of malicious behavior, but automatically extracting and classifying this information remains challenging because relevant evidence may appear both in unstructured natural language and in structured behavioral relationships. This study investigates whether combining neural semantic representations with explicit symbolic malware knowledge can improve classification of behaviors described in malware reports. Using the publicly available MalwareTextDB corpus, 2,975 annotated malware action events were extracted from 39 Advanced Persistent Threat reports, and 1,796 single-label instances from six malware capability categories were selected for evaluation. Three systems were compared: a neural semantic baseline using pretrained Sentence-BERT embeddings, a symbolic rule-based baseline operating on annotated actions, objects, and modifiers, and a hybrid neuro-symbolic model combining both score types through weighted fusion. Evaluation was performed using report-separated validation and held-out test sets. On the 298-instance test set, the neural baseline achieved 46.3% accuracy and a macro-F1 score of 0.421, while the symbolic baseline achieved 52.7% accuracy and 0.555 macro-F1. The hybrid model achieved the highest overall accuracy at 56.4% and a macro-F1 score of 0.553. Symbolic augmentation corrected 53 neural classification errors while introducing 23 new errors. These results suggest that explicit malware-behavior knowledge can complement general-purpose neural semantic representations, although the strong symbolic-only performance indicates that hybridization does not

automatically outperform structured domain knowledge. The findings support further investigation of adaptive neuro-symbolic methods for textual malware behavior analysis.

Keywords - Neuro-Symbolic AI, Malware Analysis, Threat Intelligence, Sentence Embeddings, MalwareTextDB, Cybersecurity

1. INTRODUCTION

Malware analysis is a central component of cybersecurity defense, but malicious software continues to evolve in ways that make automated analysis difficult. Contemporary malware can employ obfuscation, multiple stages of execution, varied communication mechanisms, and other behaviors that complicate both detection and characterization. As a result, researchers have increasingly explored machine learning and artificial intelligence techniques for identifying and classifying malicious behavior. Graph-based machine-learning approaches, for example, have demonstrated that structural representations such as function-call graphs can capture useful information about malware families (Wu et al., 2023). More recently, large language models and other neural language-processing techniques have also been applied across cybersecurity tasks including vulnerability detection, malware analysis, and intrusion detection (Xu et al., 2025).

A significant amount of useful malware information is also contained outside executable files themselves. Threat-intelligence reports produced by security researchers

describe malware capabilities, communication patterns, propagation mechanisms, information-stealing behavior, and other operational characteristics in natural language. However, the volume and diversity of these reports can make it difficult for analysts to rapidly identify relevant behavioral information. Lim et al. (2017) identified this challenge in the development of MalwareTextDB, noting that natural language processing could help extract malware actions and capabilities from lengthy security reports. Their dataset provides 39 annotated APT reports in which malware-related language is connected to structured semantic information based on the Malware Attribute Enumeration and Characterization (MAEC) vocabulary.

Neural language representations provide one approach to processing this unstructured information. Sentence-BERT, for example, maps natural-language sentences into semantically meaningful vector representations that can be compared using cosine similarity (Reimers & Gurevych, 2019). Such embeddings support efficient semantic comparison and retrieval but do not explicitly represent relationships such as which malware action operates on a particular object or which behavioral objective an action serves. In cybersecurity, these distinctions can be important because superficially similar language may describe different malicious behaviors. Communication with a remote server, for instance, may represent command-and-control activity in one context and data exfiltration in another.

Symbolic representations offer a complementary mechanism for representing such information. Rather than relying solely on similarity learned from data, symbolic approaches encode explicit entities, relationships, rules, or constraints. MalwareTextDB itself contains this type of structured information through annotated Actions, Subjects, Objects, Modifiers, and relations among them (Lim et al., 2017). Prior malware-analysis research has likewise shown the value of explicit structural representations, including function-call graphs and symbolic program-analysis methods. Wu et al. (2023), for example, showed that enriching function-call graph representations improved IoT malware-family classification on a dataset containing more than 108,000 malware binaries.

The complementary characteristics of neural and symbolic approaches motivate **neuro-symbolic artificial intelligence**, which seeks to integrate learned neural representations with explicit structured knowledge. Recent systems such as SymRAG demonstrate this principle by incorporating symbolic, neural, and hybrid processing

pathways within a unified framework. SymRAG dynamically routes queries among these pathways according to query complexity and system conditions, illustrating how different reasoning mechanisms can contribute to the same task rather than being treated as mutually exclusive alternatives (Hakim et al., 2025).

Despite these developments, much of the existing malware-classification literature operates directly on binaries, system calls, or graph representations of executable code, while neural language-processing research more commonly focuses on unstructured text. Comparatively less work has examined whether **structured malware knowledge extracted from threat-intelligence language can directly complement neural semantic representations of that same text**. MalwareTextDB provides an opportunity to investigate this question because each annotated malware event contains both a natural-language description and an explicit structured representation of its behavioral relationships.

Accordingly, this study investigates the following research question:

Can the integration of symbolic malware-behavior knowledge with neural semantic representations improve the classification of behaviors described in malware threat reports?

To address this question, a proof-of-concept neuro-symbolic framework was developed using MalwareTextDB. The original annotations were transformed into 2,975 action-level malware events, from which 1,796 single-label instances belonging to six malware capability categories were selected for experimental evaluation. Three systems were compared: a **neural semantic baseline** using pretrained sentence embeddings, a **symbolic baseline** using explicit rules over structured action, object, and modifier relationships, and a **hybrid neuro-symbolic system** combining the two sources of evidence through weighted score fusion. Entire threat reports were kept within individual data partitions to reduce document-level information leakage.

On the held-out test set of 298 instances, the neural baseline achieved **46.3% accuracy and a macro-F1 score of 0.421**. The symbolic system achieved **52.7% accuracy and 0.555 macro-F1**, while the hybrid model achieved the highest overall accuracy at **56.4%** and a macro-F1 score of **0.553**. Qualitative error analysis further showed that symbolic

augmentation corrected 53 predictions missed by the neural system while introducing 23 new errors. These findings suggest that explicit malware-behavior relationships can provide useful information that is not consistently captured by general-purpose semantic embeddings, although the strong performance of the symbolic-only baseline also demonstrates that neural-symbolic integration does not automatically outperform well-aligned structured knowledge.

The primary contributions of this study are:

1. **A proof-of-concept neuro-symbolic framework for malware behavior analysis** that represents the same threat-report event through both natural-language semantic embeddings and explicit behavioral relationships.
2. **An event-level transformation of MalwareTextDB** that connects threat-report context with structured malware actions, objects, modifiers, and capability annotations for experimental analysis.
3. **A comparative evaluation of neural, symbolic, and hybrid approaches** across six malware capability categories using report-separated validation and held-out test sets.
4. **An empirical and qualitative analysis of neural-symbolic interaction**, identifying cases in which symbolic knowledge corrects semantic classification errors as well as cases in which overlapping security concepts introduce new errors.

The remainder of this paper is organized as follows. Section 2 reviews related research in machine-learning-based malware analysis, symbolic and graph-based security analysis, language models and retrieval-augmented systems, and neuro-symbolic AI. Section 3 describes MalwareTextDB and the preprocessing procedure used to construct the experimental dataset. Section 4 presents the neural, symbolic, and hybrid methodologies. Section 5 reports the experimental results, followed by discussion in Section 6. Section 7 describes the limitations of the proof-of-concept study and directions for future research, and Section 8 concludes the paper.

2. BACKGROUND & RELATED WORK

Automated malware analysis has increasingly incorporated machine learning, graph-based representations, natural language processing, and, more recently, large language models. These approaches provide different mechanisms for representing malicious behavior. Neural methods can learn patterns from high-dimensional data and natural-language descriptions, while structural and symbolic approaches explicitly encode relationships among program components, actions, and security concepts. Neuro-symbolic artificial intelligence seeks to combine these complementary properties by integrating learned neural representations with explicit symbolic knowledge (Velasquez et al., 2026). The present study draws on work in four overlapping areas: machine-learning-based malware analysis, structural and symbolic security analysis, retrieval-augmented and language-model-based cybersecurity systems, and neuro-symbolic reasoning.

2.1. MACHINE LEARNING AND STRUCTURAL REPRESENTATIONS FOR MALWARE ANALYSIS

Traditional automated malware analysis is commonly divided into **static** and **dynamic** approaches. Static analysis examines a program without executing it, while dynamic analysis observes behavior during execution. Both approaches have limitations when malware employs obfuscation, polymorphism, environment detection, or other evasion mechanisms. Consequently, machine-learning methods have increasingly been applied to malware detection and classification in order to learn patterns that extend beyond fixed signatures.

Akhtar and Feng (2022), for example, evaluated several machine-learning approaches for malware detection, including decision trees, support vector machines, and convolutional neural networks. Their results demonstrate the broader applicability of learned classifiers to malware-detection tasks, while Yadav et al. (2022) similarly review malware-analysis techniques and defensive mechanisms in IoT and Android environments. Together, this line of research illustrates the increasing use of statistical learning to identify patterns associated with malicious software.

More recent work has focused not only on the choice of classifier but also on **how malware is represented before classification**. Wu et al. (2023) investigated IoT malware classification using function-call graphs (FCGs), which represent relationships among functions within a binary.

Their approach applied graph2vec to learn vector representations of these graphs and introduced additional mechanisms for incorporating function-call semantics and literal information. On a dataset containing more than 108,000 malware binaries spanning seven CPU architectures, the enhanced graph representation improved classification accuracy across multiple classifiers, with an SVM achieving 98.88% family-classification accuracy (Wu et al., 2023).

The importance of representation is also evident in more recent few-shot malware-classification research. Chen et al. (2025) introduced TOM-Net, which combines transductive meta-learning with graph-based inference over IoT malware function-call graphs. The model uses a hierarchical GraphSAGE encoder to capture structural and semantic features while addressing limited labeled data and previously unseen malware classes. TOM-Net achieved 92.64% accuracy under its reported 5-way 10-shot closed-set configuration and an AUC of 93.59% in its open-set evaluation (Chen et al., 2025).

Although these studies analyze malware binaries rather than natural-language threat reports, they establish an important theme for the present work: **malware behavior is not represented adequately by isolated surface features alone**. Structural relationships can provide information that supports more effective downstream classification.

A related principle appears in vulnerability-detection research. Rozi et al. (2024) proposed ContextCPG, an extension of the code property graph that enriches structural code representations with contextual information such as variable names and data types. By combining graph structure with additional semantic context, their system achieved an average accuracy increase of approximately 8% compared with the original code property graph representation. This result provides further evidence that security analysis can benefit when structural information is supplemented by contextual semantics rather than treated independently.

The present study applies a similar principle at the level of **threat-intelligence language**. Instead of representing binary functions or source-code dependencies, MalwareTextDB provides both natural-language descriptions and explicitly annotated relationships among malware actions, subjects, objects, and modifiers. Lim et al. (2017) designed the corpus specifically to map textual descriptions of malware behavior to structured MAEC-based attributes, allowing

behavioral information to be represented at both linguistic and semantic levels.

2.2. SYMBOLIC AND GRAPH-BASED SECURITY ANALYSIS

While machine-learning models are effective at identifying statistical patterns, symbolic methods provide a different mechanism for malware analysis by explicitly representing program behaviors, constraints, and relationships.

Bertrand Van Ouytsel and Legay (2022) investigated malware classification using **symbolic execution** combined with graph-kernel methods. Symbolic execution explores possible program execution paths while maintaining symbolic rather than concrete variable values, enabling analysis of behaviors that may not appear during a single dynamic execution. Their approach represents system-call relationships through System Call Dependency Graphs and uses graph kernels to compare malware behavior. The work illustrates how formal program-analysis techniques can generate explicit behavioral structures that are subsequently used by machine-learning classifiers.

SEMA extends this research direction into an automated toolchain. Bertrand Van Ouytsel et al. (2023) developed SEMA as an open-source symbolic-execution framework for malware analysis built on ANGR. The system extracts API calls and their arguments, constructs System Call Dependency Graphs, and provides machine-learning modules for analyzing the resulting representations. The authors argue that symbolic execution enables deeper exploration of possible malware behaviors than analysis based on a single concrete execution, while structured graph representations make those behaviors available to downstream learning systems.

Graph-based reasoning has also been applied beyond direct malware classification. Saulaiman et al. (2025) use graph structures to automate aspects of threat analysis and risk assessment in automotive cybersecurity, demonstrating the broader utility of explicit relational representations in security workflows. Although the domain differs from malware-report classification, such work reflects a recurring cybersecurity challenge: security-relevant evidence often consists not simply of individual features but of relationships among entities, actions, systems, and consequences.

These approaches differ substantially from the symbolic system developed in the present study. The current

framework does **not** perform symbolic execution of binary code. Instead, it operates on symbolic information already encoded in MalwareTextDB's human annotations. MalwareTextDB links Actions to Subjects, Objects, and Modifiers and assigns those actions to higher-level malware capabilities and objectives (Lim et al., 2017). The shared principle is therefore not the specific use of symbolic execution, but the use of explicit and inspectable relationships to represent malware behavior.

This distinction is important because explicit representations also provide a degree of interpretability. A neural model may assign a class because two passages occupy similar positions in an embedding space, while a rule-based system can expose a relationship such as evade $\rightarrow$ detection or exfiltrate $\rightarrow$ data as evidence for its prediction. The present experiment investigates whether this type of symbolic behavioral evidence can complement a neural semantic representation of the surrounding threat-report text.

2.3. LANGUAGE MODELS AND RETRIEVAL-AUGMENTED CYBERSECURITY ANALYSIS

The emergence of transformer-based language models has created an additional research direction for automated cybersecurity analysis. A systematic literature review by Xu et al. (2025) identified applications of large language models across vulnerability detection, malware analysis, intrusion detection, and other cybersecurity tasks. The review also highlights increasing use of techniques such as fine-tuning, prompt engineering, external knowledge augmentation, and agent-based workflows.

More specialized research similarly suggests that language models can assist with malware-related analysis. Jelodar et al. (2026) survey the use of LLMs for software-security tasks including code analysis, malware analysis, and reverse engineering. Lang and Schreck (2025) experimentally incorporated LLMs into memory-forensics workflows for several malware and attack scenarios. Their results showed that reasoning-oriented models could identify useful security evidence, although the systems also produced high false-positive rates and could generate plausible but unsupported threat interpretations. The latter result is particularly relevant to systems that depend solely on learned language representations: semantically plausible output is not necessarily equivalent to correct security reasoning.

This problem has contributed to interest in **retrieval-augmented generation (RAG)**. Lewis et al. (2020) introduced RAG as a framework that combines a pretrained model's parametric knowledge with external non-parametric memory retrieved at inference time. Rather than requiring all relevant knowledge to be encoded within model parameters, retrieval allows external evidence to be incorporated dynamically. Lewis et al. found that their RAG models improved performance across several knowledge-intensive tasks and could produce more specific and factual output than a parametric-only baseline.

RAG therefore provides a mechanism for grounding model behavior in external information, but retrieval alone does not necessarily encode the explicit relationships required by specialized domains. Dense semantic retrieval is designed to identify information with similar meaning, not necessarily to enforce domain-specific rules or represent hierarchical relationships.

The distinction is similar to that underlying the neural component of the present study. Sentence-BERT maps natural-language sentences into a vector space in which semantically similar content can be compared efficiently through cosine similarity (Reimers & Gurevych, 2019). This makes embedding-based methods useful for retrieving or classifying semantically related text, but the resulting vector representations do not directly express relationships such as which action is performed on which object or what security objective that behavior serves.

Consequently, recent work has explored methods that augment neural retrieval with more explicit forms of knowledge representation. Kang et al. (2027), in a survey of neuro-symbolic RAG for mental-health applications, characterize this problem as one of information fusion: dense neural retrieval can identify semantically relevant information, while symbolic structures represent discrete relationships, hierarchies, and constraints that semantic similarity alone may not capture. Although their application domain differs from cybersecurity, the representational problem is analogous to malware threat analysis, where natural-language reports coexist with structured security concepts and behavioral relationships.

2.4. NEURO-SYMBOLIC AI AND ADAPTIVE HYBRID REASONING

Neuro-symbolic AI provides a broader framework for integrating these two forms of representation. Velasquez et al. (2026) broadly define neuro-symbolic AI as the

integration of explicit symbolic representations (including logic, knowledge graphs, and other structured domain knowledge) with neural networks capable of learning implicit representations from data. In this view, symbolic knowledge may be supplied by domain experts, extracted from data, or integrated into neural systems at different stages of processing.

One recent example of such integration is **SymRAG**, introduced by Hakim et al. (2025). SymRAG addresses the observation that not all queries require the same form or amount of computation. Rather than sending every query through an identical pipeline, the framework analyzes query complexity and system conditions and routes requests through **symbolic, neural, or hybrid pathways**. Evaluated on 2,000 queries from HotpotQA and DROP using Llama and Mistral models, SymRAG demonstrated that adaptive routing could retain strong task performance while reducing unnecessary computation (Hakim et al., 2025).

The importance of SymRAG to the present study is primarily **conceptual rather than architectural**. The system developed here does not implement query routing, an LLM generation stage, or the complete SymRAG framework. Instead, SymRAG motivates the broader premise that neural and symbolic mechanisms need not be treated as competing approaches. Each can provide different information, and hybrid processing may benefit tasks in which both semantic interpretation and explicit structured reasoning are useful.

Other neuro-symbolic work similarly demonstrates the potential of combining learned and explicit representations. For example, SR-LLM incorporates retrieval-augmented language modeling into symbolic regression so that accumulated external knowledge can contribute to the construction of interpretable symbolic expressions (Guo et al., 2025). Although this work addresses mathematical model discovery rather than cybersecurity, it reflects the growing use of LLMs and retrieval as components within systems that ultimately produce or reason over explicit symbolic structures.

2.5. RESEARCH GAP

Taken together, prior work establishes several relevant findings. Machine-learning methods can successfully identify malware patterns, and graph-based representations can preserve behavioral or structural information that may be lost in flatter feature representations (Wu et al., 2023; Chen et al., 2025). Symbolic execution and graph-based

malware-analysis systems demonstrate that explicit behavioral relationships can also provide informative and interpretable representations (Bertrand Van Ouytsel & Legay, 2022; Bertrand Van Ouytsel et al., 2023). At the same time, language models and semantic embeddings provide increasingly capable mechanisms for interpreting cybersecurity text, while RAG and neuro-symbolic architectures illustrate how neural processing can be supplemented with external or structured knowledge (Lewis et al., 2020; Hakim et al., 2025).

However, these strands are frequently evaluated separately. Malware-classification research commonly focuses on binaries, system calls, or function-call graphs, while LLM and retrieval research focuses primarily on unstructured text. Symbolic malware analysis likewise tends to operate directly on executable behavior rather than on structured information embedded within threat-intelligence reports.

MalwareTextDB creates an opportunity to examine the intersection of these approaches because the same malware event is available simultaneously as **natural-language context and explicit semantic annotations**. The corpus was designed to associate malware-report text with standardized actions, capabilities, and behavioral relationships (Lim et al., 2017). This makes it possible to compare a neural representation of report language directly with a symbolic representation of the same behavior.

Accordingly, the present study investigates whether these representations can be combined within a small neuro-symbolic malware-analysis framework. Rather than attempting to construct a full production malware-analysis system, the experiment isolates a specific question: **whether explicit symbolic malware-behavior knowledge can improve capability classification when combined with general-purpose neural sentence representations**. The study therefore extends prior work by applying neuro-symbolic integration to malware behaviors expressed in threat-intelligence text and by directly comparing neural-only, symbolic-only, and hybrid configurations under a common evaluation setting.

3. DATASET AND PREPROCESSING

This study uses **MalwareTextDB**, a publicly available corpus of annotated malware-related threat reports developed by Lim et al. (2017). MalwareTextDB was created to support natural language processing research in cybersecurity and contains 39 annotated Advanced Persistent Threat (APT) reports comprising 6,819 sentences.

The reports were drawn from the publicly available APTnotes repository, with the initial corpus consisting of reports from 2014 (Lim et al., 2017).

MalwareTextDB is particularly suitable for the present study because its annotations capture both natural-language descriptions of malware behavior and structured semantic information. Its annotation framework draws on the Malware Attribute Enumeration and Characterization (MAEC) vocabulary to associate portions of malware-report text with standardized descriptions of malware characteristics (Lim et al., 2017). The dataset defines 444 attribute labels, including 211 ActionName labels, 20 Capability labels, 65 StrategicObjectives labels, and 148 TacticalObjectives labels.

The annotation process contains three stages. First, relevant textual spans are labeled as **Action**, **Subject**, **Object**, or **Modifier**. Actions represent malware-related events, Subjects identify the initiators of actions, Objects identify recipients or entities associated with actions, and Modifiers provide additional contextual information (Lim et al., 2017). Second, explicit semantic relations connect the annotated spans through SubjAction, ActionObj, ActionMod, and ModObj relations. These links encode the relationships among actions and their associated entities (Lim et al., 2017). Finally, attribute labels are assigned to annotated actions to characterize malware behavior at multiple levels, including action names, capabilities, tactical objectives, and strategic objectives (Lim et al., 2017).

Across the corpus, MalwareTextDB contains 2,975 annotated actions, 1,778 subjects, 4,411 objects, and 1,819 modifiers. It additionally contains 2,524 capability annotations, 1,592 tactical-objective annotations, 2,004 strategic-objective annotations, and 982 action-name annotations (Lim et al., 2017). The combination of free-text threat reporting and explicit semantic annotations makes the corpus useful for the present experiment, which compares neural representations of malware-report language with symbolic representations derived from structured behavioral information.

3.1. EVENT EXTRACTION

The original MalwareTextDB distribution provides threat-report text together with BRAT-format annotation files. For the present study, these annotations were converted into an event-level tabular representation in which each annotated malware **Action** was treated as a single observation.

For every action, the preprocessing pipeline extracted the report identifier, action text, associated subject, object, modifiers, malware capability, tactical objective, strategic objective, and action-name label. The surrounding textual context was also retained. This allowed the same underlying malware behavior to be represented in two forms: natural-language context for neural semantic analysis and explicit action-entity relationships for symbolic analysis.

The resulting preprocessing pipeline produced **2,975 action-level events across all 39 reports**, matching the total number of annotated actions reported by Lim et al. (2017). Of these events, **2,524 contained a malware-capability annotation** and were therefore candidates for the supervised evaluation used in this study.

3.2. CAPABILITY SELECTION

MalwareTextDB defines 20 malware capability categories within its MAEC-based annotation framework (Lim et al., 2017). For the proof-of-concept experiment, six capability categories were selected based on their representation within the corpus and their relevance to distinct forms of malicious behavior:

- command_and_control
- infection/propagation
- machine_access/control
- anti-detection
- data_theft
- data_exfiltration

Only action events containing a single capability annotation belonging to one of these six categories were retained. This produced a final experimental dataset of **1,796 malware-behavior instances**.

The resulting dataset was class-imbalanced. command_and_control contained 515 instances and infection/propagation contained 500 instances, while machine_access/control, anti-detection, data_theft, and data_exfiltration contained 229, 212, 178, and 162 instances, respectively. Because of this imbalance, both overall accuracy and macro-averaged F1 were used during evaluation. Macro-F1 was included to ensure that performance on the smaller capability classes contributed equally to the aggregate evaluation rather than allowing the two largest categories to dominate the results.

3.3. DATA PARTITIONING

The 1,796 experimental instances were partitioned by **complete threat report rather than by individual action event**. Keeping reports intact ensured that events originating from the same source document did not appear across multiple evaluation partitions, reducing the possibility that document-specific vocabulary or context could artificially inflate test performance.

The resulting partition contained 1,150 events from 24 reports in the development subset, 348 events from seven reports in the validation subset, and 298 events from eight reports in the held-out test subset.

Table 1. Experimental data partition

Partition	Events	Reports
Development	1,150	24
Validation	348	7
Test	298	8
Total	1,796	39

The development subset was retained as part of the experimental partition, although no task-specific fine-tuning of the pretrained neural encoder was performed. The validation subset was used to select the weighting parameter used by the hybrid neuro-symbolic model, while the test subset was reserved for final evaluation.

A document-level split is also consistent with the experimental methodology of the original MalwareTextDB study. Lim et al. (2017) used all 39 annotated documents and divided them into training, development, and test sets at the document level rather than randomly distributing individual sentences across partitions.

4. METHODOLOGY

This study evaluates whether explicit symbolic knowledge can complement neural semantic representations when identifying malware behaviors described in threat-intelligence reports. A proof-of-concept framework was developed containing three experimental configurations: a **neural semantic baseline**, a **symbolic**

rule-based baseline, and a **hybrid neuro-symbolic model**.

Each system receives an action-level instance derived from MalwareTextDB and predicts one of six malware capability categories: command and control, infection/propagation, machine access/control, anti-detection, data theft, or data exfiltration.

The experimental design was conceptually motivated by neuro-symbolic systems such as SymRAG, which explore the combination of neural and symbolic processing within a unified architecture (Hakim et al., 2025). The present system does not reproduce the SymRAG architecture or implement retrieval-augmented generation. Instead, it applies the broader principle of combining continuous neural representations with explicit symbolic knowledge to a malware-behavior classification task. This simplified design was selected to evaluate the feasibility of neuro-symbolic integration within the scope of a proof-of-concept malware-analysis system.

4.1. SYSTEM OVERVIEW

Each experimental instance contains two complementary forms of information derived from MalwareTextDB. First, the original natural-language context surrounding an annotated malware action is retained for neural semantic analysis. Second, the structured **Action**, **Object**, and **Modifier** annotations associated with the event are retained for symbolic reasoning. MalwareTextDB explicitly represents relationships between actions and associated subjects, objects, and modifiers, providing a structured representation of information that also appears in natural-language form (Lim et al., 2017).

The two representations are processed independently before being combined by the hybrid system. The neural component estimates the semantic similarity between a report passage and natural-language descriptions of the six malware capabilities. The symbolic component applies explicit rules to structured action, object, and modifier fields. The hybrid model then combines the two scores to produce a final capability prediction.

Conceptually, the architecture can be represented as follows:

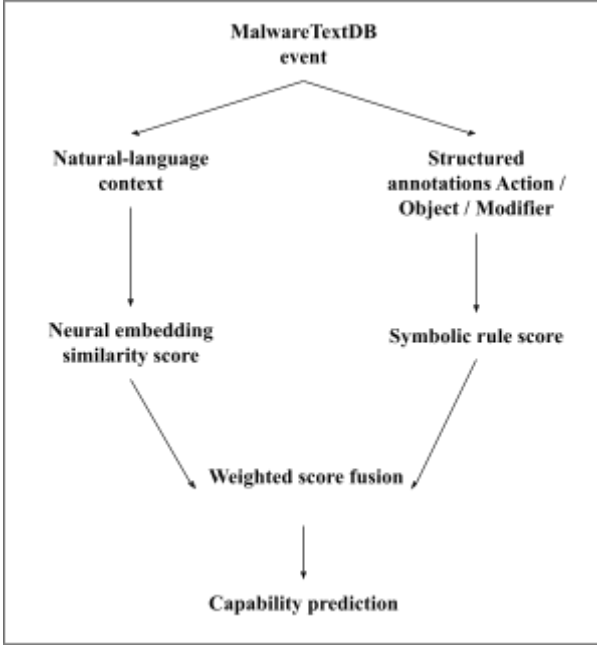


Figure 1. Hybrid neuro-symbolic system architecture

4.2. NEURAL SEMANTIC BASELINE

The neural baseline was implemented using the pretrained all-MiniLM-L6-v2 model through the Sentence Transformers library. The model was used as a sentence encoder rather than being fine-tuned on MalwareTextDB.

This approach follows the sentence-embedding framework introduced by Reimers and Gurevych (2019). Sentence-BERT produces fixed-size vector representations in which semantically related sentences can be compared efficiently using cosine similarity. The authors specifically developed the architecture to support applications such as semantic similarity search and information retrieval without requiring every possible pair of sentences to be processed jointly by a transformer model (Reimers & Gurevych, 2019).

For each of the six target malware capabilities, a short natural-language description was manually defined. For example, the description for `data_exfiltration` characterized malware that *sends, uploads, transfers, or exfiltrates stolen information from a victim environment to an attacker or external server*. Similar descriptions were constructed for command-and-control activity, infection or propagation, machine access and control, anti-detection behavior, and data theft.

Let x_i denote the natural-language context associated with malware event i , and let d_c denote the description associated with capability class c . The sentence-transformer encoder E produces vector representations for both:

$$e_i = E(x_i)$$

$$v_c = E(d_c)$$

All embeddings were normalized before comparison. The neural score for class c was therefore calculated using cosine similarity:

$$N(i, c) = \cos(e_i, v_c)$$

Because the embeddings were normalized, cosine similarity was equivalent to the dot product between the two vectors. The capability receiving the highest similarity score was selected as the neural prediction:

$$\hat{y}_N(i) = \arg_c \max N(i, c)$$

No MalwareTextDB capability labels were supplied to the neural model during prediction, and no task-specific fine-tuning was performed. The annotated capability labels were used only after prediction for evaluation.

4.3. SYMBOLIC RULE-BASED BASELINE

The symbolic component used the structured annotations associated with each MalwareTextDB action event. Whereas the neural model considered the complete surrounding textual context, the symbolic system operated primarily on the annotated **Action**, **Object**, and **Modifier** fields.

This design reflects the broader use of structured or symbolic representations in malware research while remaining substantially simpler than full symbolic execution. Prior malware-analysis systems have demonstrated that explicit representations of malware behavior can be extracted and reasoned over using symbolic execution and graph structures (Bertrand Van Ouytsel & Legay, 2022; Bertrand Van Ouytsel et al., 2023). In contrast, the symbolic representation in the present study is derived from human-annotated threat-report language rather than from exploration of executable malware code.

A set of transparent rules was manually constructed for each of the six target capabilities. Each rule contains two groups of behavioral indicators: **action terms** and **object/modifier**

terms. For example, the data-exfiltration rules include action indicators such as send, upload, exfiltrate, transfer, and transmit, together with contextual indicators such as data, file, remote server, network, HTTP, and FTP. Anti-detection rules contain action indicators such as hide, evade, bypass, disable, and obfuscate, together with contextual indicators such as antivirus, detection, sandbox, and evidence.

Representative rules are shown in Table 2.

Table 2. Representative symbolic rules

Capability	Example action indicators	Example object/modifier indicators
Command and control	connect, communicate, beacon, contact, receive	C2, controller, remote server, botnet
Infection/propagation	infect, spread, deliver, drop, install, exploit	phishing, attachment, payload, victim, document
Machine access /control	control, access, execute, run, open	remote access, shell, backdoor, host, system
Anti-detection	hide, evade, bypass, disable, obfuscate	antivirus, detection, sandbox, evidence
Data theft	steal, collect, capture, extract, search	credential, password, document, keystroke
Data exfiltration	send, upload, exfiltrate, transfer, transmit	data, file, remote server, network, HTTP

For an event i and capability c , the symbolic rule system first tests whether the annotated action matches one of the action indicators for that capability. A match contributes two points. The object and modifier fields are then searched for a capability-specific contextual indicator, which contributes

an additional two points. If both types of evidence are present, a one-point bonus is added. Thus, the raw symbolic score is:

$$R(i, c) = 2I_A(i, c) + 2I_O(i, c) + I_A(i, c)I_O(i, c)$$

where I_A equals 1 when an action rule matches and 0 otherwise, and I_O equals 1 when an object or modifier rule matches and 0 otherwise.

Because the maximum possible score is five, the value is normalized to a range between zero and one:

$$S(i, c) = \frac{R(i, c)}{5}$$

The symbolic prediction is the capability receiving the largest rule-based score. For evaluation of the standalone symbolic baseline, events for which no symbolic rule produced any evidence were treated as having no symbolic prediction rather than being assigned a capability arbitrarily.

This intentionally simple rule design provides a transparent symbolic baseline in which the evidence contributing to each prediction can be inspected directly.

4.4. HYBRID NEURO-SYMBOLIC MODEL

The hybrid system combines the semantic similarity score generated by the neural component with the normalized score generated by the symbolic component.

For each event i and candidate capability c , the combined score is calculated as:

$$H(i, c) = \alpha N(i, c) + (1 - \alpha)S(i, c)$$

where $N(i, c)$ represents the neural semantic score, $S(i, c)$ represents the symbolic score, and α controls the relative contribution of the two components.

The final hybrid prediction is:

$$\hat{y}_H(i) = \arg_c \max H(i, c)$$

An α value approaching 1 places greater emphasis on neural semantic similarity, while a value approaching 0 places greater emphasis on symbolic rules. Consequently, the framework permits symbolic evidence to modify a neural prediction without necessarily replacing the semantic representation entirely.

For example, a sentence describing malware that “*extracts the information*” may contain broad contextual language that produces an ambiguous semantic prediction. However, an annotated extracts action combined with an information object produces explicit symbolic evidence for data_theft. Conversely, symbolic rules may introduce errors when the same actions occur in multiple behavioral contexts. For example, network communication with a remote server may represent either command-and-control activity or data exfiltration depending on the purpose of the communication. These interactions were examined further through qualitative error analysis.

4.5. VALIDATION AND FUSION-WEIGHT SELECTION

The hybrid weighting parameter α was selected exclusively using the validation subset. Candidate values from 0.0 through 1.0 were evaluated in increments of 0.1.

Because the six capability classes are imbalanced, **macro-averaged F1 score** was used as the primary selection criterion. Accuracy was used as a secondary criterion when comparing configurations. The validation experiments selected:

$$\alpha = 0.8$$

resulting in a final hybrid model consisting of an 80% neural contribution and a 20% symbolic contribution.

The selected value was fixed before evaluation on the held-out test set. No changes to the symbolic rules or fusion parameter were made in response to test-set performance.

This separation between validation-based parameter selection and held-out evaluation was intended to prevent the final test results from directly influencing model configuration.

4.6. EVALUATION PROCEDURE AND METRICS

System performance was evaluated on the held-out test subset containing 298 action events from eight reports. Three configurations were compared:

1. **Neural baseline:** prediction using only sentence-embedding similarity.
2. **Symbolic baseline:** prediction using only explicit action/object/modifier rules.

3. **Hybrid model:** prediction using the validation-selected combination of neural and symbolic scores.

Two aggregate evaluation metrics were used: **accuracy** and **macro-averaged F1 score**. Accuracy measures the proportion of events for which the predicted capability matches the MalwareTextDB capability annotation:

$$Accuracy = \frac{\text{Number of correct predictions}}{\text{Total number of predictions}}$$

Because the target classes contain substantially different numbers of examples, macro-F1 was also calculated. Macro-F1 independently computes an F1 score for each capability and then averages the six values, preventing the performance of the two largest classes from dominating the aggregate result.

Class-specific precision, recall, and F1 were additionally calculated for the hybrid system. A confusion matrix was generated to identify recurring misclassification patterns among the six malware behaviors.

Finally, a qualitative error analysis compared the neural and hybrid predictions on an event-by-event basis. Two categories were examined: cases in which symbolic augmentation corrected an incorrect neural prediction and cases in which augmentation changed a correct neural prediction into an incorrect one. This analysis was used to identify conditions under which explicit symbolic information was beneficial as well as situations in which broad rule patterns introduced ambiguity.

The experiment was implemented in Python using pandas and NumPy for data processing, sentence-transformers for neural sentence encoding, scikit-learn for dataset partitioning and evaluation metrics, and Matplotlib for visualization. A fixed random seed of **42** was used during report-level partitioning to support reproducibility.

5. RESULTS

The neural, symbolic, and hybrid neuro-symbolic systems were evaluated on the held-out test subset of **298 malware-behavior events drawn from eight threat reports**. Performance was measured using overall accuracy and macro-averaged F1 score. The hybrid model used the validation-selected fusion weight of $\alpha = 0.8$, corresponding to an 80% neural contribution and a 20% symbolic contribution.

5.1. OVERALL SYSTEM PERFORMANCE

Table 3 summarizes the performance of the three experimental configurations.

Table 3. Performance of the neural, symbolic, and hybrid systems on the held-out test set.

System	Accuracy	Macro-F1
Neural semantic baseline	0.463	0.421
Symbolic rule-based baseline	0.527	0.555
Hybrid neuro-symbolic model	0.564	0.553

The neural semantic baseline correctly classified **46.3%** of the test instances and achieved a macro-F1 score of **0.421**. The symbolic baseline produced higher performance on both measures, achieving **52.7% accuracy** and a macro-F1 score of **0.555**.

The hybrid neuro-symbolic model achieved the highest overall accuracy at **56.4%**, corresponding to 168 correct predictions out of 298 test instances. In comparison, the neural baseline correctly classified 138 instances. The integration of symbolic information therefore resulted in a net increase of **30 correctly classified test events** relative to the neural-only system.

The hybrid model also substantially improved macro-F1 compared with the neural baseline, increasing from **0.421 to 0.553**. However, the symbolic-only system achieved a slightly higher macro-F1 score of **0.555**. The difference between the symbolic and hybrid macro-F1 scores was small, indicating that the symbolic component contributed strongly to balanced performance across the six target capability classes. The hybrid system nevertheless achieved the highest overall accuracy.

During validation, the fusion parameter $\alpha = 0.8$ produced a macro-F1 score of **0.521**, compared with **0.484** for the neural-only configuration. The selected value was retained unchanged for final test evaluation.

5.2. CLASS-LEVEL PERFORMANCE

Table 4 reports precision, recall, and F1 score for each malware capability under the hybrid neuro-symbolic model.

Table 4. Class-level performance of the hybrid neuro-symbolic system on the held-out test set.

Malware capability	Precision	Recall	F1	Support
Command and control	0.77	0.60	0.68	78
Infection/propagation	0.59	0.51	0.54	85
Machine access/control	0.50	0.59	0.54	29
Anti-detection	0.71	0.40	0.51	30
Data theft	0.54	0.64	0.59	39
Data exfiltration	0.36	0.65	0.46	37
Macro average	0.58	0.56	0.55	298
Weighted average	0.61	0.56	0.57	298

The strongest class-level result occurred for **command and control**, with an F1 score of **0.68** and precision of **0.77**. The system also performed comparatively well on **data theft**, which achieved an F1 score of **0.59**.

Performance was more uneven for other behavior categories. **Anti-detection** achieved relatively high precision at **0.71**, but recall was only **0.40**, indicating that many anti-detection events were assigned to other capability classes. In contrast, **data exfiltration** exhibited the opposite pattern. Its recall was relatively high at **0.65**, while precision was only **0.36**. This indicates that the hybrid system identified a substantial portion of true exfiltration instances but also over-predicted the class for behaviors belonging to other categories.

5.3. CONFUSION MATRIX ANALYSIS

Figure 2 presents the confusion matrix for the hybrid neuro-symbolic model.

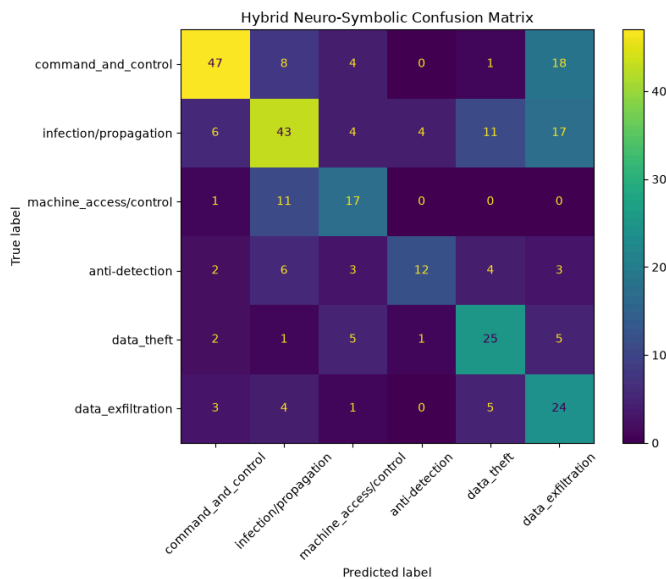


Figure 2. Confusion matrix for the hybrid neuro-symbolic malware-behavior classifier on the held-out test set ($n = 298$). Rows represent ground-truth capability labels and columns represent predicted capability labels.

The matrix shows that **47 of 78 command-and-control instances** were classified correctly. The most frequent error for this class was prediction as data exfiltration, which occurred in **18 cases**. Similarly, **43 of 85 infection/propagation instances** were classified correctly, while 17 were predicted as data exfiltration and 11 as data theft.

For **machine access/control**, 17 of 29 instances were correctly identified, while 11 were classified as infection/propagation. This suggests substantial overlap between language describing malware installation, deployment, access, and subsequent control of the compromised system.

The model correctly classified **12 of 30 anti-detection instances**, making anti-detection the class with the lowest recall. Data theft was correctly identified in **25 of 39 cases**, while **24 of 37 data-exfiltration events** were correctly classified.

A notable pattern is the tendency of the hybrid model to predict **data exfiltration** for behaviors belonging to several other classes. In addition to the 24 correct exfiltration

predictions, 18 command-and-control events and 17 infection/propagation events were assigned to this category. This pattern is consistent with the relatively low precision and high recall observed for data exfiltration.

5.4. EFFECT OF SYMBOLIC AUGMENTATION

To examine how the symbolic component influenced the neural predictions, the neural and hybrid outputs were compared at the individual-event level.

Symbolic augmentation **corrected 53 instances** that had been classified incorrectly by the neural semantic baseline. At the same time, it changed **23 correct neural predictions into incorrect hybrid predictions**. The resulting net improvement of 30 predictions is consistent with the increase in overall accuracy from 46.3% to 56.4%.

Several corrected cases contained direct action-object relationships that provided strong symbolic indicators of the correct malware capability. Examples included:

- exfiltrated combined with data, which corrected a neural data_theft prediction to data_exfiltration;
- extracts combined with information, which corrected an anti-detection prediction to data_theft;
- evade combined with detection, which corrected a command_and_control prediction to anti-detection;
- search combined with certain types of data, which corrected an anti-detection prediction to data_theft;
- references to C&C domains, which corrected an infection/propagation prediction to command_and_control; and
- obfuscate combined with malware characteristics, which corrected an infection/propagation prediction to anti-detection.

These examples demonstrate cases in which explicit action and object information provided behavioral cues that were not captured sufficiently by semantic similarity alone.

However, symbolic augmentation also introduced errors when rule indicators were shared across multiple malware behaviors. For example, references to HTTP requests, remote servers, email, domains, and sending information may occur during both **command-and-control** and **data-exfiltration** activity. In several cases, the neural model correctly recognized command-and-control context, while

symbolic indicators associated with network communication shifted the hybrid prediction toward data exfiltration.

Similar ambiguity occurred between infection/propagation and machine access/control. Actions such as installing a backdoor or deploying a malicious component may describe both the mechanism through which a system becomes infected and the functionality that subsequently provides remote access. These overlapping behavioral contexts contributed to several of the observed hybrid errors.

6. DISCUSSION

The results provide initial evidence that explicit symbolic malware knowledge can complement neural semantic representations when analyzing behaviors described in threat-intelligence reports. Relative to the neural-only baseline, the hybrid model increased test accuracy from **46.3% to 56.4%** and macro-F1 from **0.421 to 0.553**. At the individual-instance level, symbolic augmentation corrected 53 neural errors while introducing 23 new errors, producing a net increase of 30 correct classifications. However, the symbolic-only baseline achieved a marginally higher macro-F1 score than the hybrid system, at 0.555 compared with 0.553. Consequently, the results do not indicate that the hybrid approach uniformly dominates either individual component. Rather, they suggest that neural and symbolic representations contribute different forms of information, and that the effectiveness of their integration depends on the behavioral context being analyzed.

6.1. COMPLEMENTARY ROLE OF SYMBOLIC KNOWLEDGE

The improvement over the neural baseline indicates that semantic similarity alone did not capture all of the information needed to distinguish malware capabilities. Sentence-BERT was designed to place semantically similar sentences near one another in an embedding space, enabling efficient comparison through cosine similarity (Reimers & Gurevych, 2019). However, semantic similarity does not explicitly encode the security-specific relationship between an action and its associated target or context. Two sentences may use similar network-related vocabulary while describing substantially different malware objectives.

The symbolic representation provided a second source of evidence based on explicitly structured malware behaviors. MalwareTextDB associates actions with subjects, objects, and modifiers and then maps those actions to higher-level malware attributes (Lim et al., 2017). In the present

experiment, this structure allowed patterns such as evade combined with detection, or exfiltrate combined with data, to contribute direct evidence toward a particular capability even when the broader surrounding sentence produced an ambiguous neural representation.

This finding is consistent with a broader line of cybersecurity research emphasizing the value of explicit structural information. Wu et al. (2023), for example, represented IoT malware using function-call graphs and graph embeddings, demonstrating that structural and semantic information within malware binaries can support family classification. Similarly, Rozi et al. (2024) combined graph-based source-code representations with contextual information in ContextCPG and reported improved vulnerability-detection performance compared with the original graph representation. Although these systems operate on executable or source-code structure rather than threat-report text, they support the broader idea that security analysis can benefit from combining contextual information with explicit relationships rather than relying on a single representation alone.

The present findings also align conceptually with neuro-symbolic architectures such as SymRAG. Hakim et al. (2025) separate symbolic, neural, and hybrid processing into distinct pathways rather than assuming that one reasoning strategy is optimal for every query. Their architecture dynamically chooses among the three according to query characteristics and system conditions. The current proof of concept uses a substantially simpler fixed-weight fusion mechanism, but the experimental results similarly indicate that the neural and symbolic components possess different strengths.

6.2. CASES IN WHICH SYMBOLIC INFORMATION HELPED

The qualitative error analysis illustrates why structured information was useful in many cases. Several neural errors involved sentences whose complete context contained vocabulary associated with multiple cybersecurity behaviors. When the action and object were considered explicitly, however, the intended capability became substantially clearer.

For example, the neural model classified an event containing the action exfiltrated and object data as data theft, while symbolic augmentation correctly shifted the prediction to data exfiltration. Likewise, an event containing extracts and information was changed from anti-detection to

the correct data-theft class, and an evade-detection relationship corrected a command-and-control prediction to anti-detection. In such cases, the action-object relationship carries more specific behavioral information than the overall semantic similarity between the surrounding text and a short capability description.

This observation is particularly relevant to the design of MalwareTextDB itself. Lim et al. (2017) explicitly created separate action, subject, object, and modifier annotations in order to represent relationships within malware descriptions rather than merely identifying security-related vocabulary. The authors describe these relations as a mechanism for making the links among behavioral elements explicit. The present results suggest that even relatively simple rules operating on this structure can recover information that a general-purpose semantic encoder may overlook.

Related malware-analysis work has similarly explored explicit behavioral structures. Bertrand Van Ouytsel and Legay (2022) combined symbolic execution with system-call dependency graphs and graph-kernel methods for malware classification, while Bertrand Van Ouytsel et al. (2023) presented SEMA, a symbolic-execution toolchain that extracts system-call information and constructs structured behavioral representations for subsequent machine-learning analysis. These approaches differ substantially from the symbolic rules used in the present study: they analyze executable behavior through formal program-analysis techniques, whereas the current system reasons over human-annotated threat-report language. Nevertheless, both approaches reflect the usefulness of representing malware behavior through explicit relationships rather than through surface-level textual or syntactic features alone.

6.3. SOURCES OF ERROR AND BEHAVIORAL AMBIGUITY

Symbolic augmentation was not uniformly beneficial. Although 53 neural errors were corrected, 23 initially correct neural predictions became incorrect after symbolic information was introduced. The confusion matrix indicates that much of this behavior arose from **overlapping security vocabulary and behavioral concepts**.

The clearest example was data exfiltration. The hybrid model achieved a recall of 0.65 for this class but a precision of only 0.36. It therefore identified many genuine exfiltration events while also incorrectly assigning the label to numerous instances from other classes. Eighteen

command-and-control instances and 17 infection/propagation instances were incorrectly classified as data exfiltration.

This pattern can be explained in part by the symbolic rules used in the experiment. Terms such as send, HTTP, server, network, and data occur naturally in exfiltration descriptions, but they are not exclusive to exfiltration. Malware communicating with command-and-control infrastructure may also make HTTP requests, contact remote servers, or transmit information. A rule system based on relatively broad indicators can therefore identify the existence of network communication without reliably determining its **purpose**.

A similar overlap occurred between infection/propagation and machine access/control. An event describing the installation of a backdoor may simultaneously relate to the process by which malware establishes itself on a system and the remote-control functionality that the installed component ultimately provides. The single-label classification setting requires one capability to be selected even when the underlying behavior may plausibly relate to multiple stages of an attack.

Importantly, ambiguity of this type is not unique to the proposed system. The creators of MalwareTextDB reported substantial difficulty even during human annotation. Lim et al. (2017) reported a Cohen's kappa of 0.36 for token-label annotation and noted that complex sentences may admit multiple reasonable decompositions. They further observed that some malware attribute labels are open to interpretation and that accurate annotation requires specialized cybersecurity knowledge. These observations provide useful context for the misclassifications observed in the present study: distinctions among malware capabilities can themselves be semantically and operationally ambiguous.

6.4. INTERPRETING THE SYMBOLIC-ONLY PERFORMANCE

One of the more notable findings is that the symbolic baseline achieved a macro-F1 score of **0.555**, slightly exceeding the hybrid system's score of **0.553**. This result is important because it prevents a simplistic interpretation in which adding a neural model automatically improves a symbolic system.

The symbolic rules were developed around a small number of relatively broad malware capabilities and operated on annotations that had already been created by human

cybersecurity annotators. As a result, the structured action, object, and modifier fields contained highly task-relevant information. The neural baseline, by contrast, relied on a general-purpose pretrained sentence encoder and six manually written capability descriptions. No cybersecurity-specific fine-tuning was performed. The symbolic component therefore had access to a representation more directly aligned with the target task.

At the same time, the hybrid model achieved the highest **accuracy**, suggesting that the neural component remained useful for some events where rule-based evidence alone was insufficient. The two aggregate metrics therefore describe slightly different aspects of system behavior. The symbolic baseline produced the strongest balanced performance across classes, while the hybrid model produced the largest total number of correct classifications.

This result also reinforces the importance of evaluating neuro-symbolic approaches against both neural and symbolic baselines. Without the symbolic-only comparison, the increase from 46.3% to 56.4% accuracy might be interpreted primarily as evidence for successful neural-symbolic integration. The symbolic result shows instead that a substantial portion of the improvement originated from the strength of the structured representation itself.

6.5. IMPLICATIONS FOR NEURO-SYMBOLIC MALWARE ANALYSIS

These results show that a neuro-symbolic approach is feasible for this task, but they also show that the integration strategy matters. A fixed weighted combination successfully improved overall classification relative to the neural baseline, but the error analysis shows that symbolic evidence should not necessarily receive the same influence for every event.

One way to address this in future work would be to borrow SymRAG's adaptive-routing idea. Rather than applying a single neural-symbolic mixture to every input, Hakim et al. (2025) dynamically route queries through symbolic, neural, or hybrid pathways depending on the characteristics of the task. A malware-analysis system could potentially apply a similar principle. Events containing highly discriminative symbolic relationships, such as *evade* → *detection* or *exfiltrate* → *data*, might receive greater symbolic weight. Events containing ambiguous indicators such as *send*, *server*, or *HTTP* could instead rely more heavily on contextual neural representations.

A richer symbolic representation could also incorporate more of the information already available in MalwareTextDB. The present proof of concept uses primarily actions, objects, and modifiers, while the corpus additionally contains tactical objectives and strategic objectives that provide progressively more detailed descriptions of malware capabilities (Lim et al., 2017). Future systems could represent these relationships as a knowledge graph or hierarchical reasoning structure rather than as independent keyword rules.

Similarly, the neural component could be strengthened through domain adaptation. The current model performs zero-shot comparison between threat-report passages and short capability descriptions. Sentence-BERT provides an efficient mechanism for representing and comparing semantic content (Reimers & Gurevych, 2019), but the present experiment does not adapt the encoder specifically to malware terminology or threat-intelligence language. A cybersecurity-specific or task-fine-tuned embedding model may therefore capture distinctions that the general-purpose representation failed to resolve.

Overall, the research question can be answered cautiously in the affirmative. **Within this proof-of-concept experiment, integrating symbolic malware-behavior information with neural semantic representations improved classification relative to the neural-only baseline, particularly in overall accuracy and balanced class performance.** However, the symbolic-only results demonstrate that the benefit should not be attributed simply to the presence of a hybrid architecture. Instead, the results indicate that structured domain knowledge is highly informative for malware-behavior analysis and that neural representations can complement this knowledge when the two are integrated appropriately. More sophisticated symbolic representations and adaptive integration strategies are needed before broader conclusions can be drawn about the effectiveness of neuro-symbolic malware-analysis systems.

7. LIMITATIONS AND FUTURE WORK

The results of this study should be interpreted within the scope of a proof-of-concept experiment. Several limitations affect the generalizability of the findings and identify opportunities for future research.

7.1. LIMITATIONS

The first limitation concerns the **size and age of the dataset**. MalwareTextDB contains only 39 annotated APT reports, and the original corpus was constructed from reports published in 2014 (Lim et al., 2017). Although the dataset provides unusually detailed semantic annotations of malware behavior, the threat landscape has changed substantially since the corpus was created. New malware families, attack techniques, infrastructure patterns, and terminology may therefore be underrepresented or absent. For that reason, the MalwareTextDB results should not be treated as representative of current threat-reporting data.

The experiment also considered only **six of the 20 malware capability categories** defined by MalwareTextDB. The selected classes were chosen to provide a manageable proof-of-concept task and sufficient representation within the available data, but they do not encompass the full range of malware behavior represented by the corpus. The original MalwareTextDB framework includes 20 capability labels as well as more detailed tactical and strategic objectives (Lim et al., 2017). Expanding the system to a larger label space would create a substantially more difficult classification task and may reveal additional interactions between neural and symbolic representations.

The **symbolic component** is also deliberately simple. The symbolic system relies on manually constructed keyword and action-object rules. Although this design makes predictions transparent and easy to inspect, it cannot capture the full meaning of complex malware behaviors. Generic terms such as send, server, HTTP, and install may appear in multiple security contexts, resulting in incorrect rule activation. The confusion between command-and-control and data exfiltration observed in the test results demonstrates this limitation.

The symbolic component also depends on structured annotations that were originally created by human annotators. MalwareTextDB itself documents the difficulty of this task. Lim et al. (2017) reported a Cohen's kappa of 0.36 for token annotations and noted that complex sentences can support multiple reasonable annotation structures. The authors additionally observed that malware labeling requires specialized cybersecurity knowledge and that some capability labels are open to interpretation. The ground-truth labels used in the present experiment should therefore not be interpreted as perfectly unambiguous representations of malware behavior.

The neural component has limitations as well. The experiment uses a **general-purpose pretrained sentence encoder** without cybersecurity-specific fine-tuning. Sentence-BERT was developed to produce semantically meaningful embeddings that support efficient similarity comparison using cosine similarity (Reimers & Gurevych, 2019). However, the model used in this experiment was not adapted specifically to malware reports or threat-intelligence terminology. As a result, distinctions that are meaningful to a cybersecurity analyst may not be represented strongly in the embedding space.

In addition, the neural system performs zero-shot classification by comparing each threat-report context with **manually written natural-language descriptions of the six capability classes**. The wording of these descriptions may influence the resulting similarity scores. Alternative descriptions, multiple descriptions per capability, or descriptions derived directly from formal cybersecurity vocabularies could produce different results.

The experiment was also evaluated using a **single report-level train/validation/test partition**. Although keeping complete reports within individual partitions reduces document-level leakage, the resulting scores may depend partly on which reports were assigned to each subset. Repeated cross-validation or evaluation across multiple report-level splits would provide stronger evidence regarding the stability of the observed results.

Relatedly, no statistical significance tests were performed. The approximately ten-percentage-point accuracy difference between the neural and hybrid systems is meaningful within this test set, but the present experiment cannot establish whether a similar improvement would consistently occur across other report samples or datasets.

Finally, the scope of the study is limited to **malware behavior described in threat-intelligence text**. The system does not analyze executable binaries, perform dynamic malware execution, inspect memory images, or conduct symbolic execution of malware code. The term *malware analysis* in this study therefore refers specifically to automated interpretation and classification of malware behaviors described in textual threat reports rather than direct reverse engineering of malicious software.

7.2. FUTURE WORK

Future work could address these limitations by first evaluating the approach on a **larger and more**

contemporary threat-intelligence corpus. Incorporating recent malware reports would provide a better test of whether the proposed neuro-symbolic approach generalizes to current terminology, attack techniques, and malware behaviors.

The symbolic representation could also be expanded beyond manually constructed keyword rules. MalwareTextDB already includes hierarchical information through capability, strategic-objective, and tactical-objective annotations (Lim et al., 2017). The dataset describes strategic and tactical objectives as increasingly detailed representations of malware capabilities, providing a natural basis for richer symbolic reasoning. These relationships could be represented as a **knowledge graph** in which actions, objects, capabilities, and objectives are connected explicitly. Such a representation could distinguish between superficially similar actions by incorporating their broader behavioral context.

A future system could also incorporate established cybersecurity knowledge sources such as malware ontologies or structured threat frameworks. Connecting report-level evidence to standardized security concepts could reduce reliance on manually chosen keywords and allow the symbolic component to reason over more complex relationships.

The neural component could be improved through **domain-specific adaptation**. Fine-tuning a sentence encoder on cybersecurity or threat-intelligence text may better capture distinctions between closely related behaviors such as command-and-control communication and exfiltration. Task-specific contrastive learning could additionally encourage passages describing the same malware capability to occupy similar regions of the embedding space while separating behaviorally distinct classes.

Future experiments should also investigate alternatives to the fixed weighted fusion strategy used in this study. The current hybrid model applies the same 80% neural and 20% symbolic weighting to every event. However, the error analysis indicates that some events contain strong, highly discriminative symbolic evidence, while others contain ambiguous rules that should be trusted less. An **adaptive routing or confidence-based fusion mechanism** could assign greater weight to symbolic reasoning when explicit behavioral relationships are present and rely more heavily on neural context when symbolic evidence is weak or ambiguous. This direction is conceptually consistent with

neuro-symbolic architectures such as SymRAG, which explore adaptive use of neural and symbolic processing rather than applying a single reasoning strategy universally (Hakim et al., 2025).

Evaluation should also be expanded beyond a single split. Repeated report-level cross-validation, statistical significance testing, and external validation on a second malware-report corpus would provide stronger evidence regarding the robustness of the approach. Future work could additionally examine retrieval-oriented metrics or analyst-centered evaluations, such as whether the system helps security analysts identify relevant malware behaviors more quickly or produces explanations that are easier to verify.

Finally, future research could move beyond single-label capability prediction toward **multi-label and hierarchical malware-behavior analysis**. Individual malware actions may contribute simultaneously to infection, persistence, command-and-control, and information theft. A system capable of representing multiple related behaviors and their relationships would more closely reflect the complexity of real malware operations than the single-capability classification task used in this proof of concept.

8. CONCLUSION

This study investigated whether symbolic malware-behavior knowledge could complement neural semantic representations when classifying behaviors described in malware threat-intelligence reports. Using MalwareTextDB, a proof-of-concept framework was developed that compared three approaches: a neural semantic baseline based on pretrained sentence embeddings, a symbolic rule-based baseline operating on structured action, object, and modifier annotations, and a hybrid neuro-symbolic model combining both sources of evidence.

The results demonstrate that structured malware knowledge contributed meaningful information beyond neural semantic similarity alone. On the held-out test set, the neural baseline achieved **46.3% accuracy and a macro-F1 score of 0.421**, while the symbolic baseline achieved **52.7% accuracy and 0.555 macro-F1**. The hybrid system achieved the highest overall accuracy at **56.4%** and a macro-F1 score of **0.553**. Symbolic augmentation corrected 53 predictions missed by the neural system while introducing 23 new errors, resulting in a net increase of 30 correctly classified test instances.

These findings suggest that the research question can be answered cautiously in the affirmative: **explicit symbolic malware-behavior knowledge can improve the classification of threat-report behaviors when combined with general-purpose neural semantic representations.** At the same time, the symbolic-only system's strong performance shows that the improvement cannot be attributed simply to the presence of a hybrid architecture. Rather, the results emphasize the value of structured domain knowledge and the importance of how neural and symbolic information are integrated.

The error analysis further demonstrated that the two representations have complementary strengths. Symbolic action-object relationships were especially useful when behavioral cues were explicit, such as exfiltrating data or evading detection. However, broad indicators associated with network communication, installation, and information transfer also introduced ambiguity between related capabilities such as command-and-control, data exfiltration, infection, and machine access.

Overall, this work demonstrates the feasibility of applying a neuro-symbolic approach to textual malware behavior analysis and provides a foundation for more sophisticated systems. Future work should explore richer knowledge representations, cybersecurity-specific neural models, adaptive neural-symbolic weighting, larger and more contemporary threat-intelligence datasets, and multi-label behavioral analysis. Such extensions may allow neuro-symbolic methods to better capture the complex and overlapping relationships that characterize real-world malware activity.

ACKNOWLEDGMENTS

The author thanks Dr. Huihui Wang for her guidance and mentorship throughout this project, and the CRA DREAM program for supporting this research experience.

REFERENCES

- Akhtar, M. S., & Feng, T. (2022). Malware analysis and detection using machine learning algorithms. *Symmetry*, 14, 2304. <https://doi.org/10.3390/sym14112304>
- Bertrand Van Ouytsel, C.-H., Crochet, C., The Dam, K. H., & Legay, A. (2023). Tool paper—SEMA: Symbolic execution toolchain for malware analysis. In S. Kallel, M. Jmaiel, M. Zulkernine, A. Hadj Kacem, F. Cuppens, & N. Cuppens (Eds.), *Risks and Security of Internet and Systems: 17th International Conference, CRiSIS 2022, Revised Selected Papers* (Lecture Notes in Computer Science, Vol. 13857, pp. 62–68). Springer. https://doi.org/10.1007/978-3-031-31108-6_5
- Bertrand Van Ouytsel, C.-H., & Legay, A. (2022). Malware analysis with symbolic execution and graph kernel. In H. P. Reiser & M. Kyas (Eds.), *Secure IT Systems: 27th Nordic Conference, NordSec 2022* (Lecture Notes in Computer Science, Vol. 13700, pp. 292–310). Springer. https://doi.org/10.1007/978-3-031-22295-5_16
- Chen, M.-Y., Ban, T., Cheng, S.-M., & Takahashi, T. (2025). TOM-Net: Few-shot IoT malware classification via open-set aware transductive meta-learning. In *2025 22nd Annual International Conference on Privacy, Security, and Trust (PST)*. IEEE. <https://doi.org/10.1109/PST65910.2025.11268864>
- Guo, Z., Wang, S., Tian, Y., Yang, J., Yu, H., Na, X., Kovács, L., Li, L., Ioannou, P. A., & Wang, F.-Y. (2025). SR-LLM: An incremental symbolic regression framework driven by LLM-based retrieval-augmented generation. *Proceedings of the National Academy of Sciences*, 122(52), e2516995122. <https://doi.org/10.1073/pnas.2516995122>
- Hakim, S. B., Adil, M., Velasquez, A., & Song, H. H. (2025). SymRAG: Efficient neuro-symbolic retrieval through adaptive query routing. *Proceedings of Machine Learning Research*, 284, 1–25. 19th Conference on Neurosymbolic Learning and Reasoning.
- Jelodar, H., Bai, S., Hamed, P., Mohammadian, H., Razavi-Far, R., & Ghorbani, A. (2026). Large language model (LLM) for software security: Code analysis, malware analysis, reverse engineering. *Journal of Information Security and Applications*, 98, 104390. <https://doi.org/10.1016/j.jisa.2026.104390>
- Kang, X., Ding, W., Matsumoto, K., Wang, L., Yu, H.-T., & Shi, X. (2027). Neuro-symbolic information fusion for explainable retrieval-augmented generation in mental health applications: A survey and

- challenges. *Information Fusion*, 138, 104702.
<https://doi.org/10.1016/j.inffus.2026.104702>
- Lang, J.-H., & Schreck, T. (2025). Leveraging LLMs for memory forensics: A comparative analysis of malware detection. *Digital Threats: Research and Practice*, 6(4), Article 22.
<https://doi.org/10.1145/3748263>
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33.
- Lim, S. K., Muis, A. O., Lu, W., & Ong, C. H. (2017). MalwareTextDB: A database for annotated malware articles. *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*, 1557–1567.
<https://doi.org/10.18653/v1/P17-1143>
- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, 3982–3992.
- Rozi, M. F., Ban, T., Ozawa, S., Yamada, A., Takahashi, T., & Inoue, D. (2024). Securing code with context: Enhancing vulnerability detection through contextualized graph representations. *IEEE Access*, 12, 142101–142126.
<https://doi.org/10.1109/ACCESS.2024.3467180>
- Saulaiman, M. N.-E., Kozlovsky, M., & Csilling, A. (2025). Graph-based automation of threat analysis and risk assessment for automotive security. *Information*, 16, 449.
<https://doi.org/10.3390/info16060449>
- Velasquez, A., White, L., Cooper, P., Zhao, A., & Chen, L. (2026). What is neurosymbolic AI? An overview and frontier problems. In A. Velasquez, H. H. Song, P. Ravikumar, S. S. Sastry, & S. Neema (Eds.), *Neurosymbolic AI: Foundations and Applications* (pp. 3–14). IEEE Press/Wiley.
<https://doi.org/10.1002/9781394302406>
- Wu, C.-Y., Ban, T., Cheng, S.-M., Takahashi, T., & Inoue, D. (2023). IoT malware classification based on reinterpreted function-call graphs. *Computers & Security*, 125, 103060.
<https://doi.org/10.1016/j.cose.2022.103060>
- Xu, H., Wang, S., Li, N., Wang, K., Zhao, Y., Chen, K., Yu, T., Liu, Y., & Wang, H. (2025). Large language models for cyber security: A systematic literature review. *ACM Transactions on Software Engineering and Methodology*.
<https://doi.org/10.1145/3769676>
- Yadav, C. S., Singh, J., Yadav, A., Pattanayak, H. S., Kumar, R., Khan, A. A., Haq, M. A., Alhussen, A., & Alharby, S. (2022). Malware analysis in IoT & Android systems with defensive mechanism. *Electronics*, 11, 2354.
<https://doi.org/10.3390/electronics11152354>